



Unidad 4

Series Temporales: Prophet

Facebook Prophet

ÁREAS DE APLICACIÓN Y CASOS DE USO: ENTORNOS CON
DATOS ESTRUCTURADOS

Modelo Prophet

Una vez comprendidos los modelos **ARIMA**, que se basan en la suposición de estacionariedad y requieren una cuidadosa selección de parámetros (p, d, q), surge la necesidad de contar con herramientas más flexibles y automatizadas, especialmente cuando se trabaja con series temporales que presentan estacionalidades múltiples, festivos u otras irregularidades.

En este contexto, **Facebook Prophet** se presenta como una alternativa poderosa y fácil de usar. Prophet es un modelo desarrollado por el equipo de ingeniería de Facebook diseñado específicamente para pronosticar series temporales de tipo empresarial: datos con fuertes efectos estacionales, tendencias a largo plazo y posibles interrupciones abruptas. **A diferencia de ARIMA**, Prophet adopta un enfoque aditivo en el que la serie temporal se descompone en tres componentes principales:

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

- $g(t)$: función de tendencia, modela cambios no periódicos en la serie.
- $s(t)$: cambios periódicos (por ejemplo, estacionalidad semanal o anual).
- $h(t)$: efectos de los días festivos, que pueden afectar uno o más días.
- ϵ_t : término de error, representa variaciones no explicadas por el modelo (más adelante se asume que sigue una distribución normal).



Una de sus principales ventajas es que maneja automáticamente los cambios de tendencia mediante detección de *changepoints*, y permite incorporar fácilmente variables externas como días festivos, todo ello sin requerir un conocimiento estadístico profundo por parte del usuario.

ARIMA vs Prophet

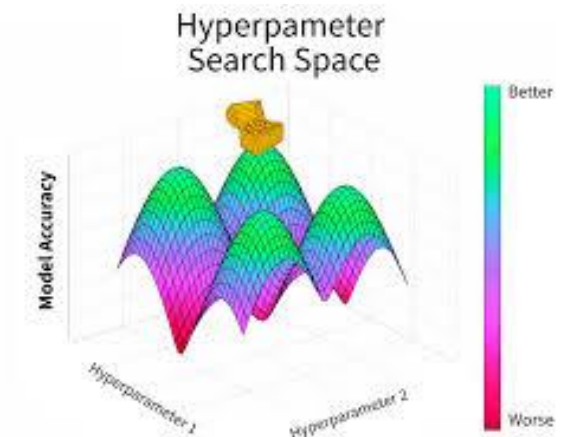


Característica	ARIMA	Prophet (Facebook)
Tipo de modelo	Estadístico clásico	Modelo aditivo basado en descomposición
Supone estacionariedad	Sí	No necesariamente
Tratamiento de tendencia	Requiere diferenciación manual	Automático, con detección de <i>changepoints</i>
Tratamiento de estacionalidad	Incluida como parte del modelo SARIMA	Se modela de forma explícita y flexible
Incorporación de festivos	Manual y compleja	Sencilla y nativa
Curva de aprendizaje	Alta (requiere conocimientos estadísticos)	Baja (intuitivo y automatizado)
Visualización del modelo	Limitada	Alta, con descomposición clara de componentes
Parámetros principales	p, d, q (y P, D, Q para SARIMA)	Crecimiento, estacionalidad, festivos, <i>changepoints</i>

Parametros del Prophet

Componente	Descripción
Holidays	DataFrame que contiene los días festivos relevantes para el modelo.
Seasonality	Define si se consideran estacionalidades anuales, semanales o diarias . Puede ser <code>True</code> o <code>False</code> .
Seasonality_mode	Tipo de estacionalidad: puede ser aditiva o multiplicativa , según cómo se combinen con la tendencia.
Seasonality_prior_scale	Controla la fuerza de la estacionalidad. Valores más altos permiten mayor flexibilidad en patrones estacionales.
Holiday_prior_scale	Valores más altos permiten al modelo ajustarse más fuertemente a los efectos de los festivos .
Changepoint_prior_scale	Determina qué tan fácil es que el modelo detecte cambios en la tendencia . Un valor más alto permite cambios más bruscos.

Usar **Hyperopt o GridSearch** ayuda a automatizar la optimización de hiperparámetros, explorando un mayor número de configuraciones



¿Como implementar el Prophet?

```
m = Prophet() # Ajustar los datos.
```

```
# Recuerda que Prophet espera que las columnas se llamen "ds" y "y".
```

```
m.fit(dataset)
```

```
# Debemos crear un DataFrame que contenga las fechas para nuestra predicción.
```

```
# El parámetro periods cuenta los días siempre que la frecuencia sea 'D' (diaria).
```

```
# Vamos a hacer una predicción de 180 días, aproximadamente medio año.
```

```
future = m.make_future_dataframe(periods=180, freq='D')
```

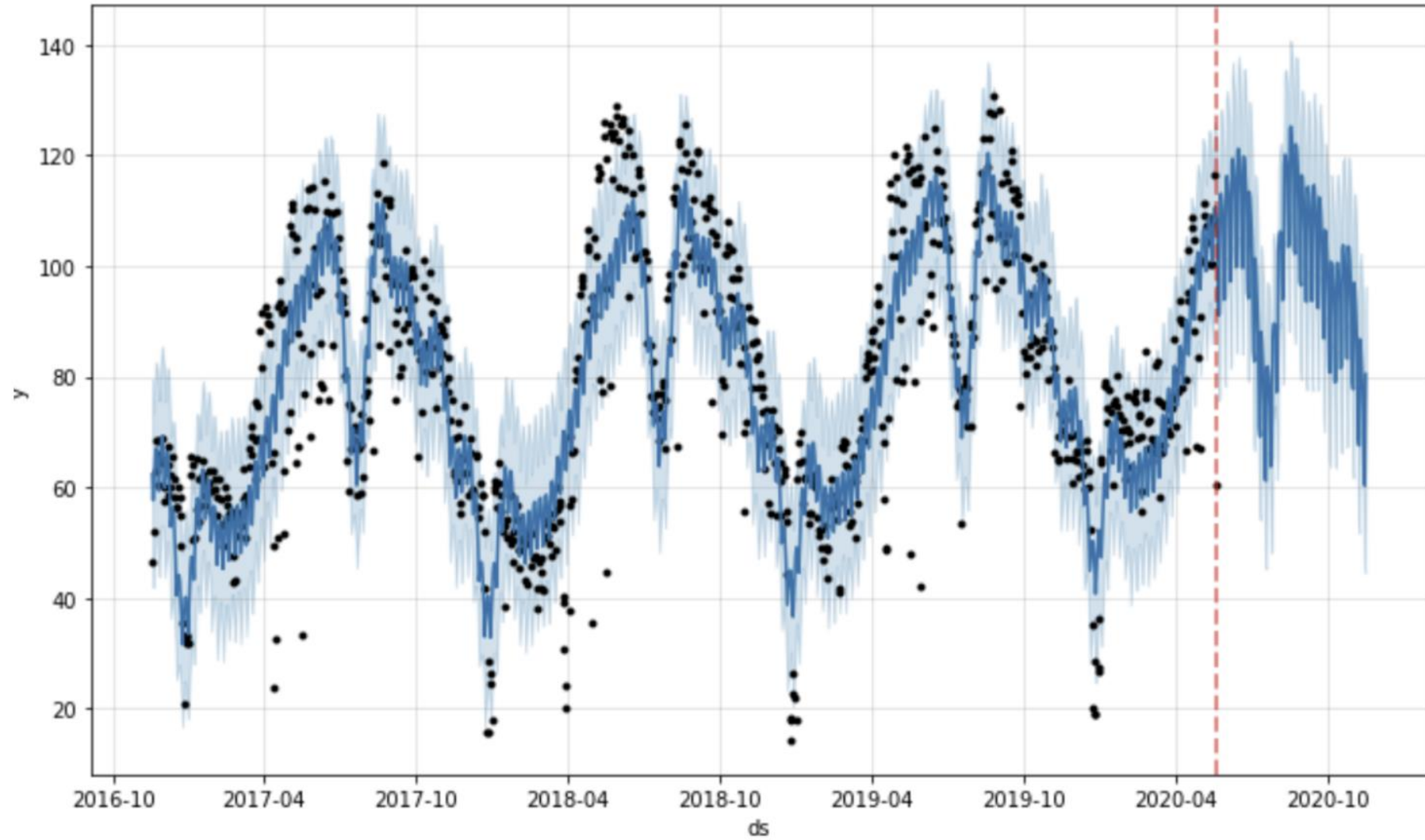
```
# Crear el objeto de predicción que  
contendrá todos los datos resultantes  
de la predicción.
```

```
forecast = m.predict(future)
```

```
# Lista los valores predichos  
con un rango inferior y superior.
```

	ds	yhat	yhat_lower	yhat_upper
1084	2020-11-12	79.240986	62.913135	94.520964
1085	2020-11-13	73.757164	58.416121	89.831205
1086	2020-11-14	60.946555	46.065980	75.921638
1087	2020-11-15	60.332186	44.588498	75.827837
1088	2020-11-16	80.458622	65.286398	96.291127

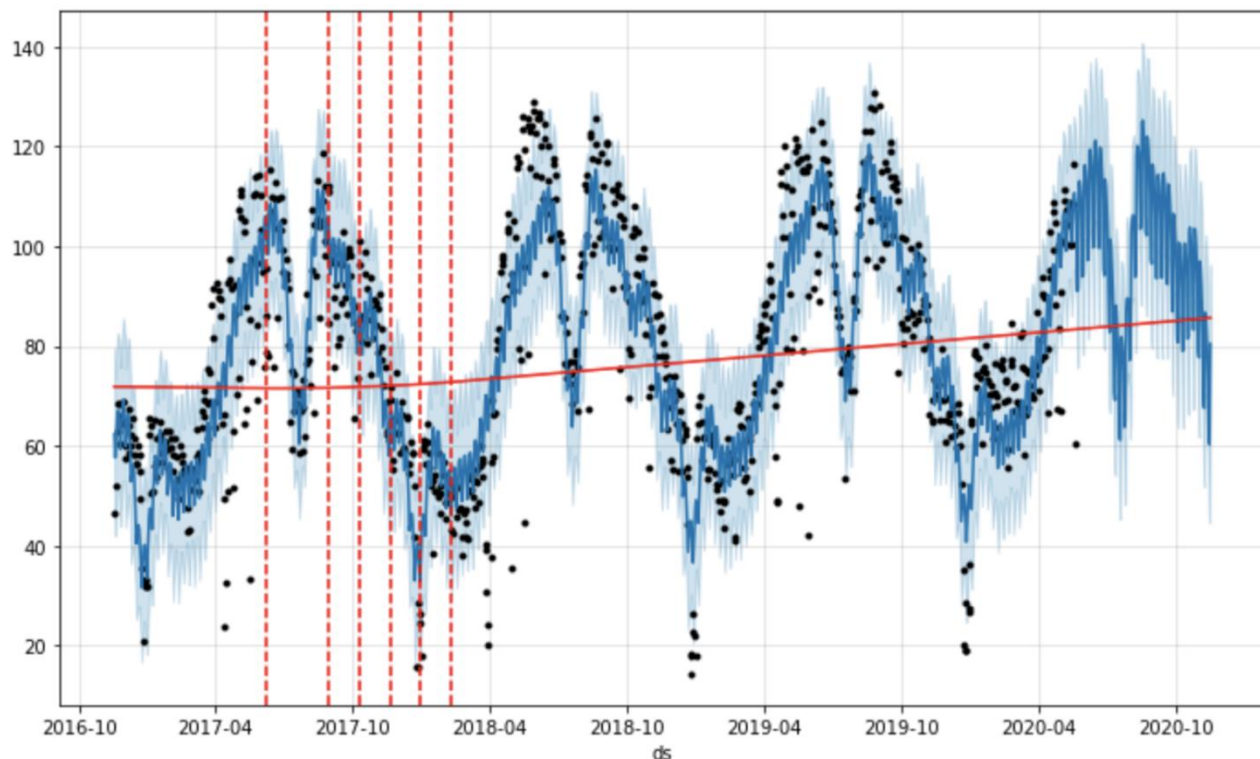
Plot del Prophet



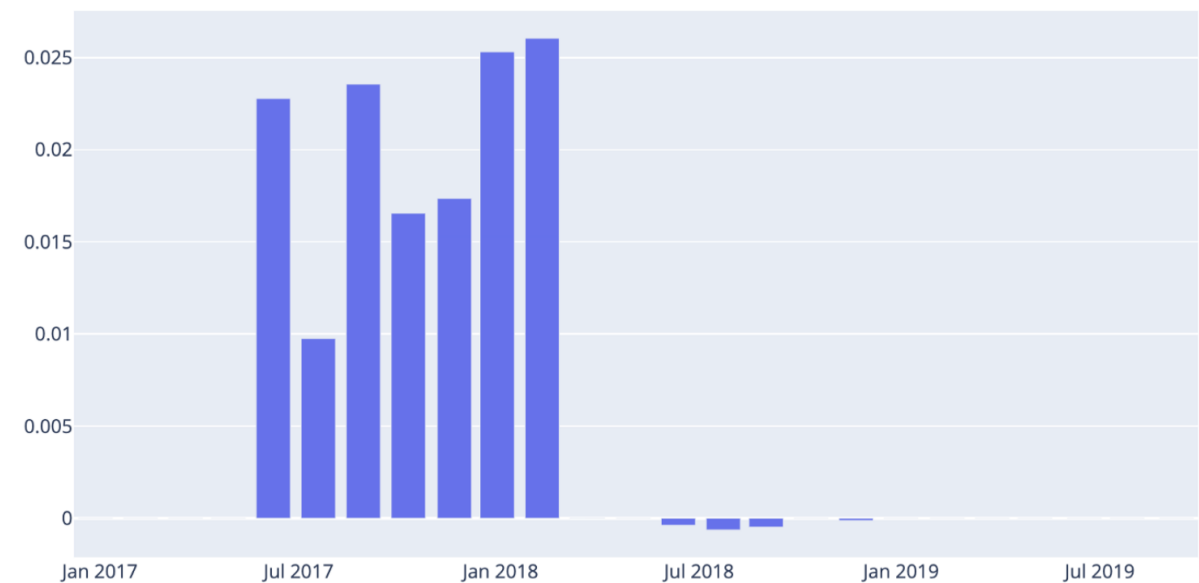
Change-Points

Prophet detecta automáticamente los posibles change points al entrenar el modelo.

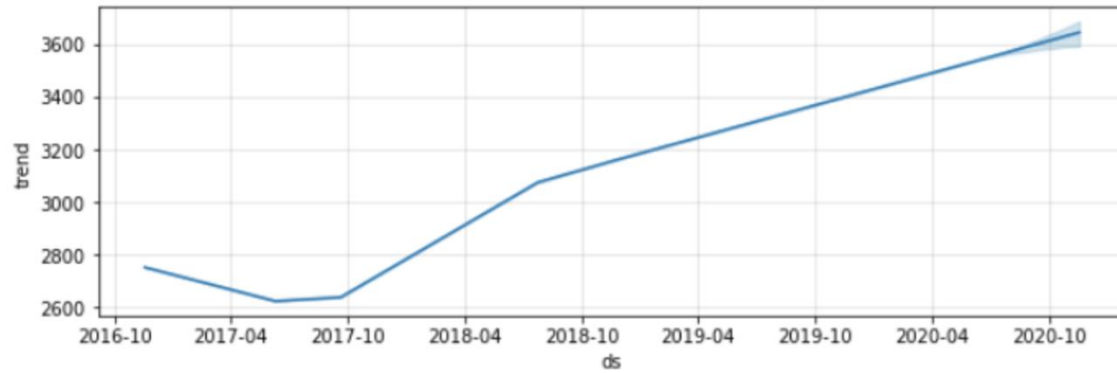
En **Prophet** (la librería de modelado de series temporales de Facebook), los change points o puntos de cambio son momentos en el tiempo donde el comportamiento de la serie temporal cambia significativamente, especialmente en la tendencia.



```
# Listing all the change points in the model
deltas = m.params['delta'].mean(0)
cp = pd.DataFrame(m.changepoints)
cp['deltas'] = deltas
fig = go.Figure()
# Create and style traces
fig.add_trace(go.Bar(x=cp['ds'], y=cp['deltas'], name='CPs',))
```



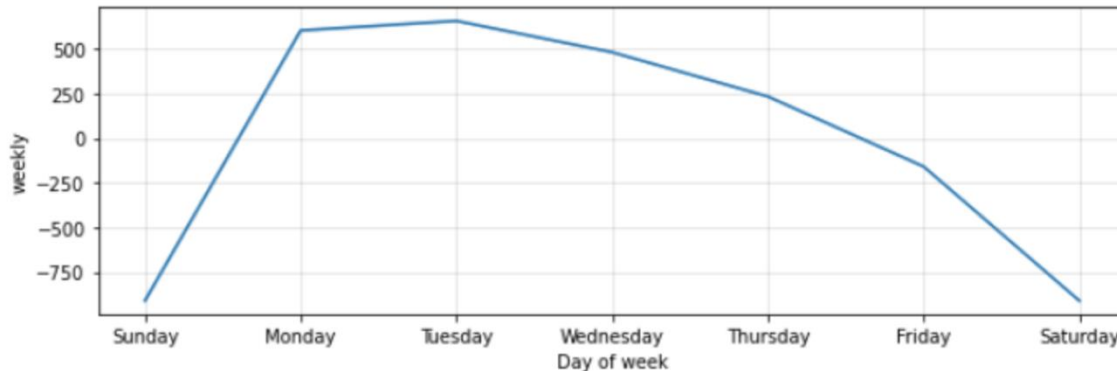
Componentes del Prophet



Representa la **tendencia general** del valor a lo largo del tiempo (ds en el eje x).

Aquí puedes ver cómo el valor general sube de forma bastante constante desde mediados de 2017 hasta finales de 2020.

La banda azul clara representa el **intervalo de confianza**, es decir, la incertidumbre del modelo.

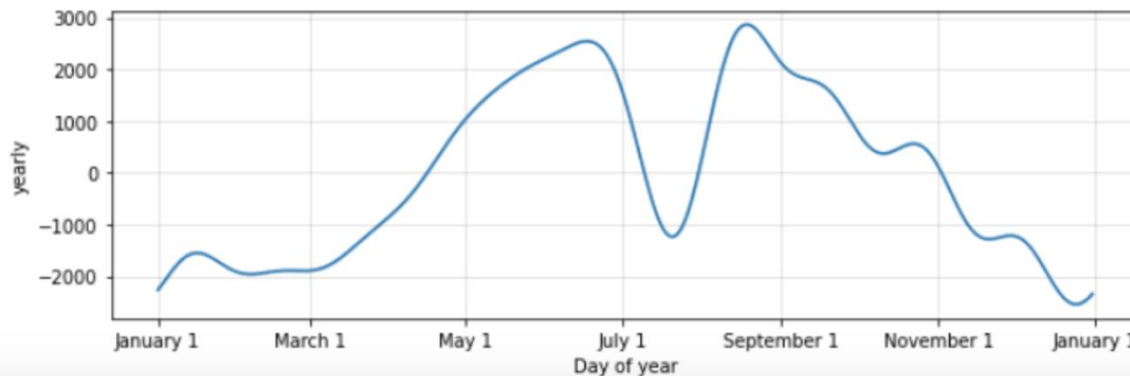


Estacionalidad semanal:

Muestra el efecto de **cada día de la semana** en el valor predicho.

En este caso:

- Lunes y martes tienen valores positivos → son días con mayor actividad o valor.
- Sábados y domingos tienen valores negativos → días con menor valor.



Estacionalidad anual:

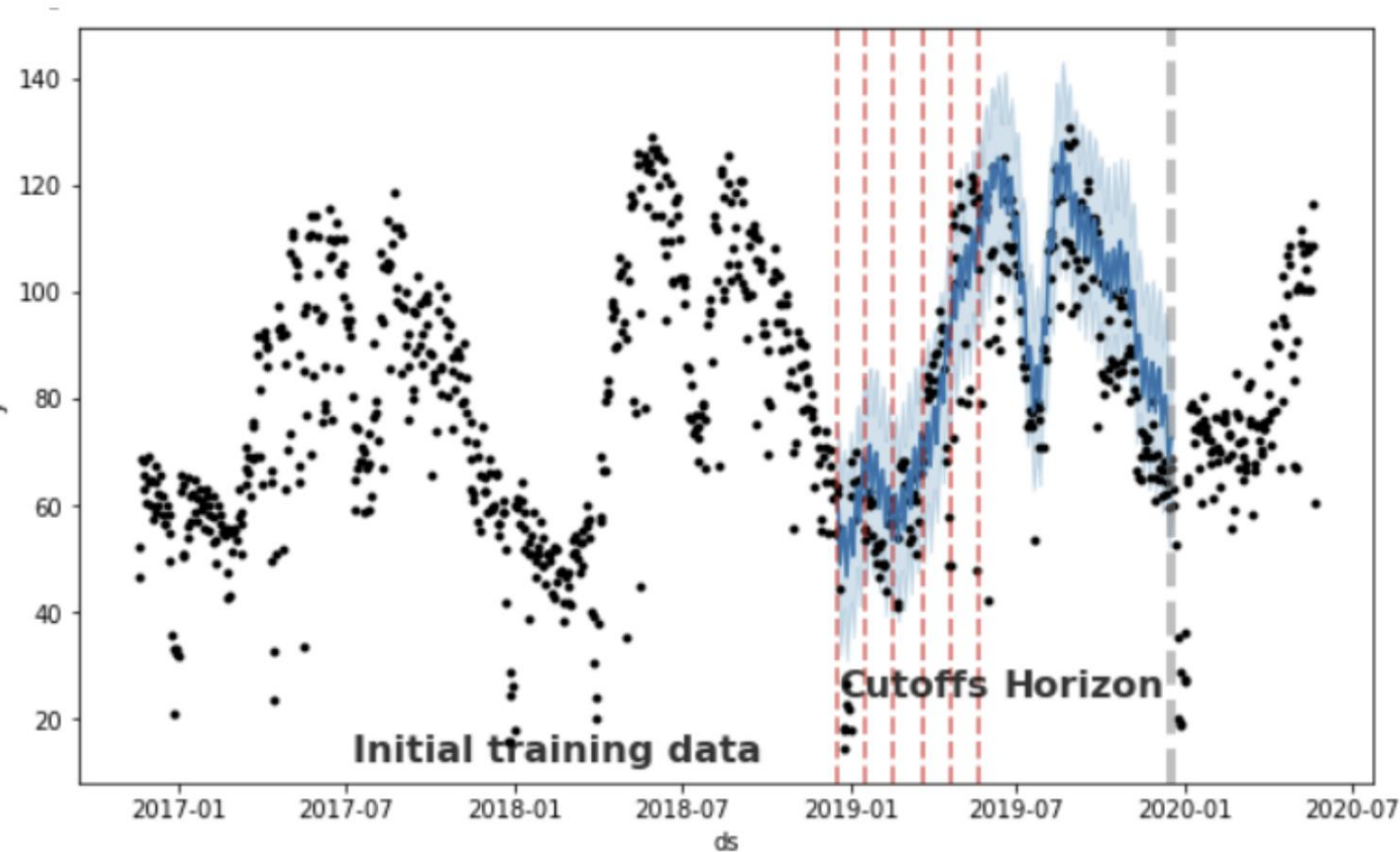
Representa el patrón estacional a lo largo del **año calendario**.

Aquí puedes ver por ejemplo:

- Un pico alrededor de **mayo a junio** y **agosto a septiembre**.
- Caídas notorias en **julio** y **diciembre a enero**.

Validación Cruzada en el Prophet

A diferencia de otros tipos de datos, **las series temporales tienen un orden cronológico**, así que no podemos mezclar aleatoriamente los datos de entrenamiento y prueba. Prophet resuelve esto con una técnica específica de **validación temporal progresiva**: se entrena el modelo en una ventana inicial de tiempo y luego se prueba en una ventana futura (el *horizon*), repitiendo esto varias veces avanzando en el tiempo.



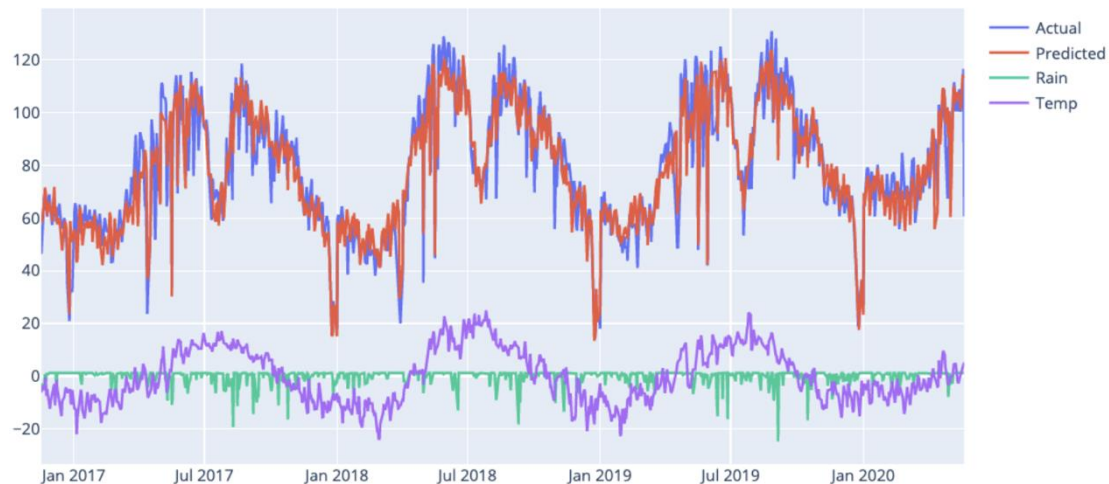
```
df_cv = cross_validation(m, initial='730 days',
                        period = '31 days', horizon = '365 days')
```



Esto significa:

- **initial**='730 days': Los **primeros 730 días** (2 años) de datos se usan para entrenar inicialmente.
- **horizon**='365 days': Después de cada entrenamiento, el modelo hace **predicciones para 1 año (365 días)** en el futuro.
- **period**='31 days': Cada **31 días**, Prophet avanza en el tiempo y hace una nueva división de entrenamiento/prueba.

Prophet con varibales exogenas



```
m = Prophet(holidays=holidays)
m.add_country_holidays(country_name='NO')
# Adding the extra weather regressors
m.add_regressor('rain')
m.add_regressor('temp')

m.fit(bikerides)
future = m.make_future_dataframe(periods=0, freq='D')
future = future.merge(bikerides, on='ds')
forecast = m.predict(future)
fig = go.Figure()
```

Prophet con varibales exogenas + hyperparameter tuning

```
m = Prophet(holidays=holidays, changepoint_prior_scale=0.01,
            changepoint_range=0.8,
            seasonality_prior_scale=0.5,
            holidays_prior_scale=1.0,
            yearly_seasonality=20,
            weekly_seasonality=5,
            seasonality_mode='additive')
```

```
m.add_country_holidays(country_name='NO')
# Adding the extra weather regressors
m.add_regressor('rain')
m.add_regressor('temp')
```

